

Elapse

You only pay what elapsed. Per-second subscriptions: protocol, merchant SDKs, docs, and webhooks.

Detailed product and build document · Furqaan Nabi · 2 September 2026 · Rank 1 of 3 (score 51/60) · committed build

COMMITTED BUILD

Track 2 · \$30,000

Bounties \$19–24K

Ceiling ~\$54K

Best if solo or 2

Chain 143 · Monad mainnet

One sentence. Elapse is Stripe Billing for things that should charge by the second — APIs, streams, GPU time, SaaS seats — settled in AUDS on Monad, with a merchant SDK, hosted docs, and signed webhooks. Demo line: you only pay what elapsed.

Product shape. Not a single consumer app. A protocol plus a developer platform. Merchants integrate in an afternoon: create a product, drop in Checkout, receive `subscription.canceled` like they already do from Stripe. Subscribers see Face ID and a ticking dollar counter — never a chain. Hearth is postponed; household money can sit on this meter later as a merchant, not as the hackathon submission.

PROBLEM

Usage is continuous. Billing is not. Stripe, App Store, and every SaaS stack still charge by the month because card networks and slow chains made anything finer uneconomic.

- Cancel on day 3, pay for 30. The meter is a lie.
- APIs, GPUs, and live streams should price by the second. They cannot, so they round up or run prepaid credits.
- Onchain “subscriptions” are wallet popups, gas, and no merchant webhook. Engineers will not replace Stripe with that.

SOLUTION

A per-second meter on Monad (300ms blocks, near-zero fees) wrapped like Stripe: protocol, SDK, docs, signed webhooks.

- Subscriber: Face ID checkout, live USD ticker, cancel mid-second, pay only elapsed time.
- Merchant: `npm install @elapse/sdk`, hosted Checkout URL, `subscription.canceled` with `seconds_elapsed`.
- No webhook spam per second. Accrue onchain; notify on start, cancel, settle, and failed payment.

1. Why this, not a demo checkout

Track 2 asks for subscriptions that charge by the second. A ticking merchant widget wins a demo. A platform wins a startup judge (and DeltaV). ArcFlow already streamed payroll; the missing piece was the merchant interface: SDK, docs, webhooks. That is how Stripe, Agora, and Privy got distribution — not a pretty one-off.

Judges include protocol founders and investors. “Here is our hosted checkout” is table stakes. “Here is `npm install`, a webhook secret, and a merchant who canceled mid-second and only paid elapsed time” is the story.

2. Product definition

What Elapse is

Three surfaces, one protocol:

SURFACE	WHO	JOB
Protocol	Monad contracts	Accrue AUD per second; cancel; settle elapsed only
Platform API	Merchants	Products, customers, subscriptions, webhook endpoints, API keys
Checkout	Subscribers	Face ID, fund, start/stop, see the live rate — no chain words
SDKs	Merchant engineers	TypeScript first, Python second, webhook signature verify
Docs	Merchant engineers	Quickstart in 10 minutes, API reference, webhook event catalog

What it is not

- Not Hearth. No household roles, no kid caps. A merchant meters one customer at a time. A house can be a customer later.
- Not x402-only. x402 upto is a stretch settlement path for API-call billing. Recurring per-second SaaS uses AccrualStream, not HTTP 402 on every tick.
- Not a webhook per second. That DDoS-es merchants and burns Envio. Accrue onchain; webhook on lifecycle and on settle batches.
- Not a wallet. Subscribers never “connect.” Merchants never paste a private key into the SDK — they use a publishable key + secret key, Stripe-style.

3. Objects (API nouns)

Mirror Stripe so merchants do not learn a new religion. Prefix ids: prod_, cus_, sub_, wh_, evt_.

OBJECT	FIELDS THAT MATTER	ONCHAIN
Merchant	name, webhook_url, webhook_secret, keys	payout address (AUD)
Product	name, rate_usd_per_second, currency=AUD	ratePerSecond wei
Customer	email / passkey id, default_payment	subscriber EOA (Mera/Privy)
Subscription	status, product, customer, started_at, canceled_at	AccrualStream id
Invoice	period, seconds, amount_settled	settle() pull
WebhookEndpoint	url, secret, events[], disabled	—
Event	type, data, created, pending_webhooks	Envio + our event log

Subscription status: incomplete → active → paused | canceled. No past_due theatre in MVP: if the pot cannot settle, fire invoice.payment_failed and pause.

4. SDKs

4.1 Packages (hackathon)

PACKAGE	LANGUAGE	MVP
@elapsed/sdk	TypeScript	Yes. Node 20+. Used in docs snippets and the demo merchant.
elapsed	Python 3.11+	Yes if time in week 5; at least <code>construct_event</code> + create subscription so it is not TS-only.
@elapsed/checkout	React	Optional. Hosted checkout URL is enough; embed is stretch.

4.2 TypeScript surface (lock this)

```
import { Elapse } from "@elapsed/sdk";

const drift = new Elapse({ secretKey: process.env.ELAPSE_SECRET_KEY });

const product = await drift.products.create({
  name: "GPU · 4090",
  rateUsdPerSecond: "0.004",
});

const session = await drift.checkout.sessions.create({
  product: product.id,
  successUrl: "https://merchant.example/ok",
  cancelUrl: "https://merchant.example/cancel",
});
// session.url → subscriber Face ID checkout

const event = drift.webhooks.constructEvent(
  rawBody, // unparsed bytes
  headers["x-elapsed-signature"],
  process.env.ELAPSE_WEBHOOK_SECRET
);
```

Also ship: `products.retrieve/list`, `subscriptions.retrieve/cancel`, `customers.retrieve`, `invoices.list`. No ORM. REST under the hood. Secret key in server environments only; publishable key only for `Checkout.js` if we add it.

4.3 Python surface (same nouns)

```
from elapsed import Elapse

drift = Elapse(secret_key=os.environ["ELAPSE_SECRET_KEY"])
event = drift.webhooks.construct_event(payload, sig, webhook_secret)
```

If Python slips, docs still show the cURL equivalents. Do not skip webhook verification in TS.

4.4 Signature

Stripe clone, on purpose:

- Header `X-Elapsed-Signature: t=unix,v1=hmac_sha256`

- Signed payload = {t}. {raw_body}
- Reject if |now - t| > 300 seconds
- HMAC-SHA256 with the endpoint's signing secret (whsec_)
- SDK compares with constant-time equality

5. Webhooks

5.1 Event catalog (MVP)

TYPE	WHEN	MERCHANT ACTION
checkout.session.completed	Subscriber finished Face ID checkout	Provision access
subscription.created	Stream opened onchain	Mark entitled
subscription.updated	Pause / resume / rate change	Sync entitlement
subscription.canceled	Cancel; elapsed seconds settled	Revoke access immediately
invoice.settled	Keeper or cancel pulled accrued AUD	Book revenue (not every second)
invoice.payment_failed	Pot empty / allowance fail	Pause product access

Do not ship invoice.tick or per-second webhooks. Accrual is onchain and in the dashboard ticker.

Merchants who want a live meter poll GET /v1/subscriptions/:id (rate + started_at → compute locally) or subscribe to a dashboard websocket we do not promise for MVP.

5.2 Delivery pipeline

1. Contract emits StreamOpened / StreamCanceled / Settled.
2. Envio HyperIndex handlers (Effect API, cache: false for delivery side effects) POST to **our** ingest URL, not the merchant's. Indexer must not know merchant secrets.
3. Platform API writes an Event, enqueues delivery jobs (retry 0s, 30s, 2m, 10m, 1h — cap 8 tries).
4. Worker POSTs JSON to merchant webhook_url with signature. Merchant returns 2xx. Timeouts 10s.
5. Dashboard shows recent deliveries: status, response code, “resend”.

Envio's documented webhook effect is the indexer → platform hop. The Stripe-grade hop (signed, retried, secret-rotated) is ours. That split is the Envio bounty *and* a real product.

5.3 Payload shape

```
{
  "id": "evt_1S2...",
  "object": "event",
  "type": "subscription.canceled",
  "created": 1756800000,
  "data": {
    "object": {
      "id": "sub_...",
      "status": "canceled",
      "seconds_elapsed": 83,
      "amount_settled": "0.33",
      "currency": "ausd",
      "product": "prod_...",
      "customer": "cus_..."
    }
  }
}
```

6. Docs (first-class, not a README)

Hosted at `docs.elapse.dev` (or Vercel `elapse-docs`). Mintlify or Nextra. Left nav:

1. Introduction — per-second money, why Monad 3ooms
2. Quickstart — API key, create product, Checkout URL, one webhook
3. Checkout — subscriber UX, success/cancel URLs
4. Subscriptions — start, pause, cancel, elapsed billing
5. Webhooks — catalog, signatures, retries, local CLI forwarder
6. SDKs — TS, Python, cURL
7. API reference — OpenAPI generated
8. Contracts — addresses, events, settle semantics
9. Test clocks — fast-forward demo rates for the hackathon video

Quickstart must be copy-pasteable in <10 minutes using testnet + a CLI that prints webhook bodies (`npx @elapse/cli listen --forward localhost:3000/webhooks`). The demo merchant in the video is that quickstart, not a special case.

Write the docs against the SDK, not the other way around. If a method is not in the SDK, it is not in the docs.

7. Subscriber checkout (the Track 2 prompt)

Hosted page: `pay.elapse.dev/c/cs_...`

1. Merchant name, product name, **live per-second rate in USD**.
2. Face ID via Privy (or Mera if Privy \$5K vs Mera \$2.5K — pick one wallet layer). Recommendation: **Privy** for subscriber checkout (embedded wallet, email/passkey, \$5K bounty) unless Mera PRF is already working from other work.

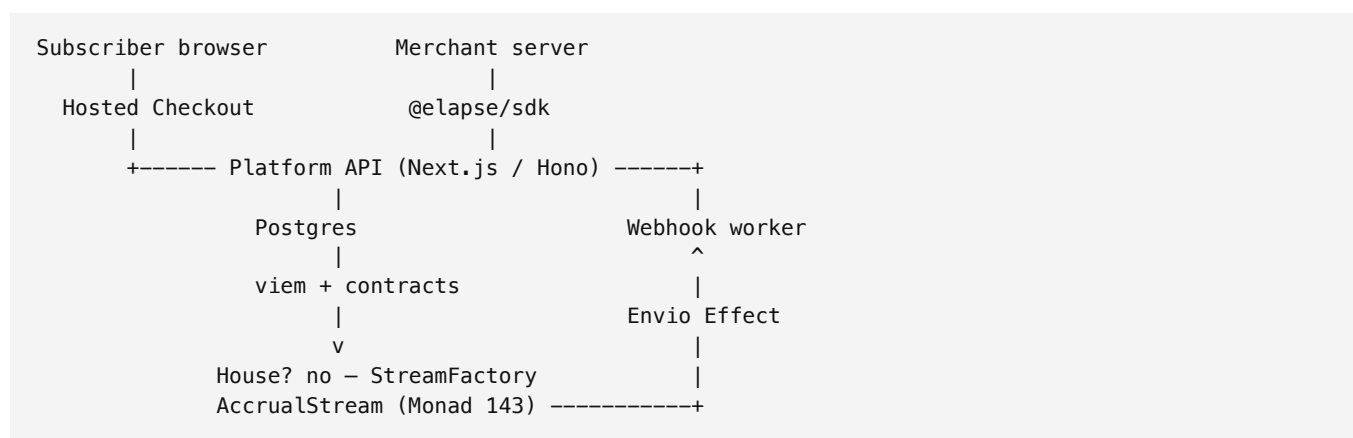
3. Fund: AUDS on Monad, or Aurora “pay from any chain” if Week 3 lands.
4. Start. Counter ticks. Cancel. “You paid 83 seconds · \$0.33.”
5. Redirect to `success_url?session_id=`

Primary UI: no Monad, gas, seed, ox. Judge mode: chain 143, 300ms ticker, contract, Envio query, webhook delivery log.

8. Merchant dashboard (thin)

Enough to not force SQL: API keys, products, live subscriptions, webhook URL + secret reveal once, delivery log, payout address. This can be the same Next.js app as Checkout with a `/dashboard` route. Do not build Stripe Atlas. Three pages.

9. Architecture



PIECE	STACK
Contracts	Solidity 0.8.24, Foundry, cloneable AccrualStream + Factory
API	Hono or Next.js Route Handlers, OpenAPI, API keys hashed at rest
DB	Postgres (merchant, products, events, delivery attempts)
Index	Envio HyperIndex on chain 143
Webhooks	Queue in Postgres + worker (good enough). No Kafka.
Checkout	Next.js, Privy, viem, 300ms ticker from rate math + started_at
Docs	Mintlify/Nextra, OpenAPI snippet
SDKs	TS published to npm (or GitHub pkg), Python if week 5
Gas	Relayer/paymaster so subscribers never hold MON
Money	AUSD. Display USD. Not yield-bearing.

Contracts

- `StreamFactory.create(merchant, subscriber, token, ratePerSecond) → clone`
- `AccrualStream`: start (escrow or pull-from-subscriber-balance), pause, cancel, settle
- Subscriber deposits AUDS into a per-sub escrow or a customer balance contract. Escrow per subscription is simpler for MVP: cancel refunds unspent.
- `settle()` batched by keeper (owner index / relay) every K seconds or on cancel. UI ticks without a tx per second.
- Events: `StreamCreated`, `StreamStarted`, `StreamPaused`, `StreamCanceled`, `Settled(seconds, amount)`

10. Demo script (3 minutes) – SDK-first

1. **0:00–0:35 Docs.** Open docs. Quickstart. npm install. Create product. Print Checkout URL.
2. **0:35–1:10 Checkout.** Phone. Face ID. Start. Ticker. Cancel at ~15s. “Paid elapsed only.”
3. **1:10–1:50 Webhook.** Terminal running `elapse listen` plus a tiny merchant server. Show `subscription.canceled` JSON with `seconds_elapsed`. Show signature header. Show 2xx.
4. **1:50–2:20 Failed pay.** Empty balance. `invoice.payment_failed`. Merchant server logs “revoke access.”
5. **2:20–3:00 Judge mode.** 300ms blocks, Envio, AUDS, Privy, contract. One line: “Stripe’s meter, Monad’s gas.”

The hero is the webhook payload, not a DeFi chart. Rehearse two devices + a laptop for docs/terminal.

11. Bounties (claim language)

BOUNTY	CASH	HOW ELAPSE HITS IT
Agora Cross-Border Payments	\$10,000	Subscriber funds from another chain via Aurora into AUDS escrow; merchant receives AUDS. Docs page: “Accept any-chain, bill in dollars.”
Aurora Intents	\$5,000	Checkout deposit address per customer/session.
Privy!	\$5,000	Checkout embedded wallet / passkey. Pick Privy over Dynamic.
Chainlink CRE	\$3,000	Optional: CRE workflow attests usage for metered APIs (seconds of GPU). Do not block Week 1 on CRE.
Envio	\$1,000	HyperIndex + Effect API as the canonical stream→event ingest for webhooks.

Skip Mera if Privy is the checkout. Skip Kuru/Perpl. Skip Cleanverse. Confirm stacking on the platform.

12. Six-week plan

Week 1 — Meter is real (kill gate)

- `AccrualStream` + Factory on Monad testnet. Start, tick mathematically, cancel, settle elapsed.
- One HTML page: merchant counter + subscriber cancel.

- Envio indexes StreamCanceled and Settled.

Kill criterion. If AUSD (or a stand-in test USD on Monad) cannot start→cancel→settle elapsed by end of Week 1, drop Agora and keep the stream in mock USD — still Elapse. If the stream itself cannot cancel mid-second correctly, the project is dead; do not pivot to a monthly subscription wrapper. Do not switch to Hearth in week 1 unless two people appear and the meter is already done.

Week 2 — API + webhook skeleton

- API keys, products.create, checkout.sessions.create (even if checkout is ugly).
- Event log + signed POST to a ngrok URL. Retry once.
- @elapse/sdk with constructEvent. Unit test the HMAC.

Week 3 — Checkout + Privy + docs shell

- Hosted checkout, Face ID, ticker, cancel, success URL.
- Docs: Introduction, Quickstart, Webhooks. Fake merchant repo in examples/saas.
- Gas sponsor. Envio → ingest → worker.

Week 4 — Platform complete

- Dashboard: keys, webhook deliveries, resend.
- CLI listen/forward. Full event catalog from §5.1.
- Aurora or labeled any-chain deposit. OpenAPI published into docs.

Week 5 — Polish SDKs and money

- Mainnet factory, real AUSD if accessible.
- Python SDK or freeze TS-only and document cURL.
- CRE only if usage attestation is a day of work, not a week.

Week 6 — Submit 13 Oct

- Video = demo script. Docs URL in the write-up. npm install from the README.
- Judges can clone examples/saas and receive a webhook. That is the bar.

13. Repo layout

```
elapse/
  contracts/      Foundry
  api/            Hono + Postgres
  indexer/        Envio HyperIndex
  worker/         webhook deliveries
  sdk/ts/         @elapse/sdk
  sdk/python/     elapse
  checkout/       Next.js hosted checkout + dashboard
  docs/           Mintlify/Nextra
  examples/saas/  the merchant in the video
  cli/           listen --forward
```

Monorepo (pnpm). One GitHub org. Docs deploy separately so a contract bug does not take docs down during judging.

14. MVP vs later

IN FOR 13 OCT	OUT
TS SDK + HMAC webhooks + retries	Per-second webhook spam
Hosted checkout + Face ID	Embedded React checkout widget
Docs quickstart + webhook catalog + OpenAPI	Multi-language SDK zoo
One product, one rate, cancel/settle	Proration catalogues, seats, tax
Envio ingest	Hearth household roles
AUSD + attempt Aurora	Yield on escrow
examples/saas	Mobile native apps

15. Risks

RISK	MOVE
Building Stripe in six weeks	Freeze the SDK surface in §4.2. Anything not listed does not exist.
Docs drift from code	Quickstart is a CI script against testnet, or it is a lie. Run it in week 6.
Envio Effect double-fire (preload)	Ingest is idempotent on txHash+logIndex. Webhook worker dedupes evt ids.
Privy vs Mera time sink	Privy only. Revisit Mera after submission.
Looks like “just an indexer”	Lead demo with npm + webhook, not Envio GraphQL.

16. Judging narrative

1. Hook: “Cancel at 83 seconds. Pay 83 seconds. Your server finds out via webhook, not a cron job.”

2. Why Monad: monthly Stripe intervals exist because gas and finality made continuous billing stupid. 300ms / 600ms makes the meter honest.
3. Why us: ArcFlow streamed payroll. This is the merchant protocol for that idea, with the same integration path merchants already know.
4. Bounties: timestamps in the video for Privy checkout, Envio delivery, AUD settle, Aurora if live.
5. Next: Hearth as a Elapse merchant; GPU and API products as x402 upto on the same customer object.

Build Elapse as a developer platform. Protocol + TS SDK + hosted checkout + docs + signed webhooks. Register Track 2. Week 1 is start/cancel/settle elapsed. Week 2 is constructEvent. If those two land, the rest is product. Hearth waits.

Sources: Metropolis Track 2 copy; MIP-12; Envio HyperIndex Effect API webhooks; Agora AUD non-yield-bearing; Privy bounty; prior ranking (Elapse 51/60). Confirm bounty stacking on hackathon.monad.xyz. Work shown 13 Oct must be built in the six weeks.